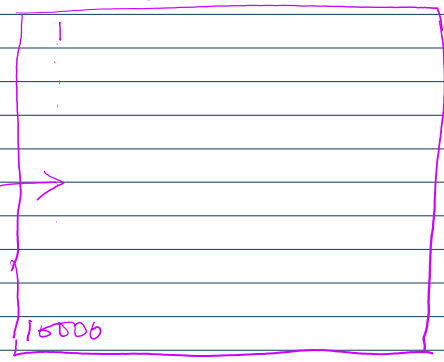
User defined Functions

04 March 2025 06:32

Lengthy program to solve complex problem.

power of compounding



Complex Problem

how to solve complex Problem?

Divide the problem into smaller units.
Solve each units.
Join all the units to solve complex problem.

Smaller unit is called functions.

What do you mean by function?

It is a group of code or code-snippet, which is solving smaller unit of complex problem.

In Python the functions are divided into two parts:

1. System defined functions: these functions are already defined in Python. For example: String functions, List function, print() function and many more.
2. User defined functions:- A user defined function in Python is a function that is created by the programmer to perform a specific task.

User defined functions allow for better code organization, reusability and modular programming.

Syntax of a User Defined function:

```
def function_name(parameters):
    """Docstring(optional): Describes what the function does. """
    #function body

    return value #(optional) returns the result to the caller of the function
```

```python
def greet():
    print('Hello, welcome to Python!')

# main code
greet()
greet()
```

Another example:

```python
def add(a, b):
    return a + b

res = add(10, 15)
print('result = ', res)
```

The return statement will return the result to the caller line.

parameters of function, which receives values from the caller of the function.

```python
def add(a, b):
    return a + b

res = add(10, 15) # call
print('result = ', res)
```

arguments or informal variable, the data of argument copy to the parameters.

```python
def add(a, b):
    return a + b
def sub(a, b):
    return a - b

# (10 + 25) - (30 - 40)
res = sub(add(10, 25), sub(30, 40))
print('result = ', res)
```

```python
"""
    Simple calulator which solves simple arithmetic  problems like
    addition(+), subtraction(-), multiplication(*), division(quotient(//), remainder(%), actual
division(/)),
    exponention(**)
"""
def add(a, b):
    return a + b
def subtract(a, b):
    return a - b
def divide(a, b):
    return a / b
def remainder(a, b):
    return a % b
def quotient(a, b):
    return a // b
def multiply(a, b):
    return a * b
def power(a, b):
    return a**b

# main code
while True:
    n1 = float(input('Enter first number: '))
    op = input('Enter operator symbol (+, -, *, /, //, **) : ')
```

```
    n2 = float(input('Enter second number: '))
    correctOp = True
    match(op):
        case "+":
            res = add(n1, n2)
        case "-":
            res = subtract(n1, n2)
        case '*':
            res = multiply(n1, n2)
        case '%':
            res = remainder(n1, n2)
        case "//":
            res = quotient(n1, n2)
        case "/":
            res = divide(n1,n2)
        case "**":
            res = power(n1, n2)
        case _:
            correctOp = False
            print('Wrong operator selected')
    if correctOp:
        print(f'Result = {res}')

    e = input("press 'x' or 'X' to exit from the calculator: ")
    if e in ('x', 'X'):
        break # terminates the loop
```

Function with multiple return value

```
def calculate(n):
    square = n ** 2
    cube = n ** 3

    return square, cube
x = int(input('Enter any integer value to find its square and cube of it: '))
s, c = calculate(x)
print('Square:', s)
print('Cube:', c)
```

Output:

```
Enter any integer value to find its square and cube of it: -3
Square: 9
Cube: -27
```

Functions with Variable number of arguments(Using args)

If you're not sure how many arguments the function will receive, you can use *args (non-keyword arguments)

```
def add(*numbers):
    total = sum(numbers)
    print("Sum:", total)

def mul(*numbers):
    p = 1
    for m in numbers:
        p = p * m
    return p
```

```
add(45, 23)
add(45, 23, 65)
add(67, 21, 19, 29)
p1 = mul(12, 4)
print("Multiplication:", p1)
p2 = mul(2, 3, 4)
print("Multiplication:", p2)
p3 = mul(3, 4, 5, 6)
print("Multiplication:", p3)
```

Output:

```
Sum: 68
Sum: 133
Sum: 136
Multiplication: 48
Multiplication: 24
Multiplication: 360
```

Function with keyword arguments(using kwargs)

- **If you want to pass arguments with key value pairs, use **kwargs.**

[name = 'Jiya', age = 12, city = "Bengaluru"]

```
def student_details(**details):
    for key, value in details.items():
        print(key, ":", value)

student_details(name='Jiya', age=12, city='Bengaluru')
```

Key value details { name = 'Jiya', age = 12, city = 'Bengaluru' }

key value Key value

Output:

```
name : Jiya
age : 12
city : Begaluru
```

Another example: with multiple records

```
def student_details(**details):
    for key, value in details.items():
        print(key, ":", value)
liName = []
liAge = []
liCity = []
while True:
    name = input('Enter your name: ')
    age = int(input('Enter your age: '))
    city = input('Enter your city: ')
    liName.append(name)
    liAge.append(age)
    liCity.append(city)

    ch = input('More records press "y" or "Y" otherwise press "n" or "N"? ')
    if ch in ('n', 'N'):
        break

for i in range(len(liName)):
    student_details(name=liName[i], age=liAge[i], city=liCity[i])
```

Output:

```
Enter your name: Shiv
Enter your age: 41
Enter your city: Kolkata
More records press "y" or "Y" otherwise press "n" or "N"? y
Enter your name: Jiya
Enter your age: 12
Enter your city: Bengaluru
More records press "y" or "Y" otherwise press "n" or "N"? y
Enter your name: Neil
Enter your age: 7
Enter your city: Begaluru
More records press "y" or "Y" otherwise press "n" or "N"? y
Enter your name: Rishabh
Enter your age: 11
Enter your city: Kolkata
More records press "y" or "Y" otherwise press "n" or "N"? n
name : Shiv
age : 41
city : Kolkata
name : Jiya
age : 12
city : Bengaluru
name : Neil
age : 7
city : Begaluru
name : Rishabh
age : 11
city : Kolkata
```

Pass statement in Function

```
def my_function():
    pass
```

Anonymous Function(Lambda Function)

- Lambda functions are single line anonymous (without a name) function.
- It is mostly used for small calculations.

```
square = lambda x: x**2
cube = lambda x: x**3
n = int(input('Enter any integer value: '))
s = square(n)
c = cube(n)
print('Square: ', s)
print('Cube: ', c)
```

Output:

```
Enter any integer value: 6
Square:  36
Cube:  216
```

- **Function with default parameters**

```
def simpleInterest(p=100, r=0.1, t=1):
    return p*r*t
si = simpleInterest()
print('Simple Interest:',si)
si = simpleInterest(1000)
print('Simple Interest:',si)
si = simpleInterest(1000, 0.12)
print('Simple Interest:',si)
si = simpleInterest(1000,0.12,3)
print('Simple Interest:',si)
```

Output:

```
Simple Interest: 10.0
Simple Interest: 100.0
Simple Interest: 120.0
Simple Interest: 360.0
```

Summary of Functions in Python

Type	**Accept Input**	**Returns Output**	**Example Use case**
Without parameters, no return.	No	No	Simple greeting message.
Which parameters, no return	Yes	No	Display user information.
With parameters, with return	Yes	Yes	Calculation, processing data.
With Default Parameters	Optional	Optional	Greeting user with default name.
Multiple Return values	Yes	Yes(multiple)	Returning square and cube of a number.
Lambda function	Yes	Yes(one-line)	Short calculations.